UNIVERSIDAD CATÓLICA BOLIVIANA

UNIVERSIDAD CATÓLICA BOLIVIANA "SAN PABLO" — Unidad Académica Regional La Paz

TEL-223 Servicios de Intranet Internet · Telecomunicaciones y Multimedia e Interactividad Digital

PRÁCTICA GUIADA 1: ENTORNO DE TRABAJO Y CONTROL DE VERSIONES

Sesión 3 · Lunes 10 de agosto de 2026 · LBRED · Docente: Dilhan Cristopher Clavijo Urrelo

Nombre	JULIO OSCAR MARTINEZ QUISPE	Puesto n.º	6 (supuesto)
Carrera	☒ Telecomunicaciones ☐ Multimedia	Fecha	10/08/2026

Qué va a lograr hoy

- El entorno mínimo verificado en su puesto: Git y, si es posible, Visual Studio Code.
- Una carpeta de trabajo que guarda el historial de todo lo que usted hace en ella.
- Un archivo README.md que describe el repositorio.
- Su primer commit registrado y el historial visible.
- Si tiene cuenta de GitHub y hay conexión: una copia privada publicada en línea.

Es una actividad formativa: no tiene calificación ni porcentaje.

Cuatro palabras, nada más

Git	Herramienta que guarda el historial de sus cambios. Funciona en su computadora.
GitHub	Plataforma en línea donde ese repositorio puede publicarse. Es opcional.
Repositorio	Una carpeta controlada por Git.
Commit	El registro de un cambio en el historial: qué cambió, quién y cuándo.

El docente explicará los demás términos durante la demostración.

El flujo que va a repetir toda la práctica

```
ARCHIVO MODIFICADO  →  git add  →  git commit  →  HISTORIAL
HISTORIAL LOCAL  →  git push  →  GITHUB  (etapa opcional)
```

Importante: Git funciona sin Internet. Solamente GitHub y git push necesitan conexión.

Qué es obligatorio y qué no: las etapas 1 a 6 son OBLIGATORIAS. La etapa 7, publicar en GitHub, es OPCIONAL: solo se hace si hay cuenta, conexión y tiempo. Su evidencia local es válida sin ella.

Llenado de ejemplo: resultados, bitácoras y casillas simulados; confirmar con la ejecución real.

Reglas de la práctica

- El docente muestra, todos ejecutan, verificamos y recién continuamos. No avance por su cuenta.
- Si termina antes, oriente a un compañero con palabras: no tome su teclado.
- Si algo falla, levante la mano. Registrar un problema es parte de la práctica.
- Instale solamente cuando el docente lo autorice, y solo Git o el editor. Nada más.
- No escriba contraseñas ni tokens en la terminal, ni datos personales en los archivos.

ETAPA 1 — Preparación y verificación del entorno (obligatoria)

No todos los puestos tienen las herramientas instaladas. Primero se verifica; solo se instala donde falte y cuando el docente lo autorice. Quien ya las tenga, verifica y avanza: no reinstale nada.

1. Marque lo que encuentra en su puesto antes de tocar nada:

Herramienta	Está disponible	No está disponible	Se instaló en clase
Git	☒	☐	☐
Visual Studio Code	☒	☐	☐
Git Bash	☒	☐	☐

2. Abra una terminal — Git Bash desde el menú Inicio, o la terminal integrada del editor si está instalado — y escriba:

```
git --version
```

Qué debe ver: una línea que empieza con git y un número de versión. El número varía según el equipo; lo importante es que responda.

Anote el resultado: Git responde ☒ · Versión detectada: 2.43.0 (ejemplo) · No responde ☐

Si no responde: Git no está instalado en ese puesto. Avise al docente y espere su autorización antes de instalar.

> **ALTO —** instale solamente si el docente lo autoriza y si el equipo lo permite sin pedir una cuenta de administrador. Si el instalador pide credenciales de administrador, deténgase: no intente evadir la restricción.

3. Instalación de Git para Windows, solo si falta y está autorizada:

- Descárguelo únicamente del sitio oficial: https://git-scm.com/download/win
- Elija el instalador para Windows de 64 bits. Si el docente le entrega el instalador en una memoria, use ese archivo.
- Ejecute el instalador y acepte las opciones predeterminadas: para esta práctica son suficientes.
- Confirme que la instalación incluya Git Bash.
- Al terminar, abra una terminal nueva y ejecute otra vez git --version.

4. Instalación de Visual Studio Code, solo si falta y está autorizada:

- Descárguelo únicamente del sitio oficial: https://code.visualstudio.com/
- Prefiera el instalador de usuario para Windows de 64 bits: normalmente no exige permisos de administrador.
- Si el instalador lo permite, marque: agregar la acción “Abrir con Code”, registrar VS Code como editor de los tipos de archivo compatibles y agregar VS Code al PATH.
- Abra VS Code y compruebe que puede abrir una carpeta.

5. Verifique que Git Bash funciona: ábralo desde el menú Inicio y ejecute otra vez git --version. Si responde, ya tiene lo indispensable, aunque VS Code no esté instalado.

6. Si no se puede instalar en su puesto:

- No intente evadir la restricción ni pedir a nadie la contraseña de administrador.

Llenado de ejemplo: resultados, bitácoras y casillas simulados; confirmar con la ejecución real.

- Anote el número de su puesto y avise al docente: quedará registrado en la ficha.
- Trabaje en pareja con un compañero cuyo puesto quedó habilitado, y roten el teclado en cada etapa.
- Siga la demostración proyectada: el contenido de la clase es el mismo para usted.

Si tiene Git pero no VS Code: puede hacer toda la práctica con Git Bash y el Bloc de notas o cualquier editor ya disponible, más el Explorador de archivos. VS Code es recomendable, no imprescindible.

Si tiene VS Code pero no Git: sin Git no hay repositorio: avise al docente y pase a trabajo en pareja hasta que su puesto quede habilitado.

PUNTO DE CONTROL 1 — su puesto quedó clasificado: habilitado, en pareja o pendiente de atención de TI. Espere la verificación del docente antes de continuar.

ETAPA 2 — Crear y abrir la carpeta de práctica (obligatoria)

7. En el Explorador de archivos, dentro de Documentos, cree la carpeta tel223-practica-git-apellido (sin espacios ni acentos).

8. Abra esa carpeta en su editor: en VS Code, Archivo → Abrir carpeta; después abra la terminal integrada, que ya arranca dentro de su carpeta. Si trabaja con Git Bash, ábralo dentro de la carpeta con el botón derecho → Git Bash Here.

Por qué importa: trabajar desde la terminal abierta dentro de la carpeta evita casi todos los errores del día.

ETAPA 3 — Crear el repositorio y configurar su identidad (obligatoria)

9. En la terminal, dentro de su carpeta, escriba:

```
git init
```

Qué debe ver: un mensaje de repositorio vacío inicializado, seguido de la ruta de su carpeta. Verifique que la ruta sea la suya.

Si falla: si la ruta no es su carpeta, avise al docente antes de seguir.

10. Configure su identidad. Escriba los dos comandos, uno por vez, con sus datos y respetando las comillas:

```
git config user.name "Nombre Apellido"
git config user.email "sucorreo@ejemplo.com"
```

Qué debe ver: nada. Estos comandos no responden: el silencio significa que funcionaron.

Por qué así: al escribir los comandos sin la opción --global, los datos quedan asociados solo a este repositorio de práctica y no a todos los trabajos futuros en ese equipo compartido. Si la aplicara a toda la computadora, el siguiente estudiante firmaría sus commits con el nombre de usted.

Verifique lo que quedó configurado:

```
git config --local --list
```

Qué debe ver: una lista corta donde aparecen user.name y user.email con sus datos.

Datos para completar la práctica (ejemplo):
Carpeta: tel223-practica-git-martinez
user.name: JULIO OSCAR MARTINEZ QUISPE
user.email: julio.martinez@example.com (correo de ejemplo)
Identidad configurada solo en este repositorio, sin --global.

Llenado de ejemplo: resultados, bitácoras y casillas simulados; confirmar con la ejecución real.

ETAPA 4 — Crear README.md (obligatoria)

11. En el editor, cree un archivo nuevo dentro de la carpeta y llámelo exactamente README.md. Créelo desde el editor, no desde el Explorador: así Windows no le agrega una extensión oculta.

12. Copie este contenido y complete sus datos:

```
# Práctica de Git — TEL-223

Estudiante: [nombre]
```
JULIO OSCAR MARTINEZ QUISPE
```
Carrera: [carrera]
```
Ingeniería de Telecomunicaciones
```
## Propósito

Practicar el registro de cambios con Git.

## Evidencia

Primer commit realizado el 10 de agosto de 2026.
```

13. Guarde el archivo con Ctrl + S. Si no lo guarda, Git no verá ningún cambio.

Nota: el diagnóstico de su equipo va en la ficha aparte, no en el README.

ETAPA 5 — Registrar el primer commit (obligatoria)

14. Consulte el estado del repositorio:

```
git status
```

Qué debe ver: el nombre de la rama y una sección de archivos sin seguimiento, donde aparece README.md.

Si falla: si dice que no es un repositorio, la terminal está fuera de su carpeta. Ábrala en el editor y use su terminal integrada.

15. Prepare el archivo y vuelva a mirar el estado:

```
git add README.md
git status
```

Qué debe ver: README.md cambió de sección: ya no está sin seguimiento, sino listo para el commit.

16. Registre su primer commit:

```
git commit -m "Crea README inicial"
```

Qué debe ver: una línea con el nombre de la rama, un identificador corto y el resumen de archivos y líneas.

Si falla: si le pide decir quién es usted, falta la configuración de la etapa 3. Ejecútela y repita el commit. No se pierde nada.

PUNTO DE CONTROL 2 — README.md quedó registrado en su primer commit. Espere la verificación del docente.

Llenado de ejemplo: resultados, bitácoras y casillas simulados; confirmar con la ejecución real.

ETAPA 6 — Revisar el historial y, si hay tiempo, un segundo commit (obligatoria)

17. Consulte el historial:

```
git log --oneline
```

Qué debe ver: una línea por commit, con su identificador corto y su mensaje.

Si la terminal parece bloqueada: presione la tecla q para salir del visor. No se colgó nada.

18. Vuelva al editor, agregue una línea a su README.md, guarde y mire el estado. Ahora el archivo aparece como modificado, no como nuevo: Git ya lo conocía.

19. Si el ritmo de la clase lo permite, registre un segundo commit:

```
git add README.md
git commit -m "Completa datos del README"
git log --oneline
```

Qué debe ver: dos líneas en el historial. Ese es el ciclo completo: cambio, reviso, preparo, registro.

PUNTO DE CONTROL 3 — Su historial es visible y su evidencia está lista. Espere la verificación del docente.

ETAPA 7 — Publicar en GitHub (OPCIONAL)

Antes de empezar: Si ya tiene una cuenta de GitHub y la conexión funciona, continúe. Si no tiene cuenta o la autenticación falla, omita esta etapa. Su evidencia local sigue siendo válida.

20. En el navegador, ingrese a su cuenta y cree un repositorio nuevo con el nombre tel223-practica-git-apellido, marcado como privado y sin agregar README ni licencia: debe quedar vacío. Si lo crea con archivos, la publicación será rechazada.

21. Copie la dirección que le muestra la plataforma y ejecute en la terminal, uno por vez:

```
git branch -M main
git remote add origin PEGUE_AQUI_LA_DIRECCION
git remote -v
git push -u origin main
```

Qué debe ver: en git remote -v, su dirección repetida dos veces; después del push, un resumen de la transferencia. Recargue la página: su README aparece publicado.

Si falla: si el envío es rechazado, el repositorio remoto no estaba vacío: cree uno nuevo sin archivos. Si la autenticación falla, no insista más de dos intentos y continúe con su evidencia local.

Si aparece la cuenta de otra persona: deténgase y avise al docente antes de publicar nada.

Verificación final entre compañeros

22. Muestre su historial y su README a un compañero, en la pantalla de su propio equipo. El repositorio es privado: no hace falta agregar a nadie como colaborador.

23. Pídale que lea sus mensajes de commit y le diga qué entiende que usted hizo. Si el mensaje no se entiende, anote cómo debería redactarlo la próxima vez. No modifique todavía el historial.

Verificado por (nombre del compañero)	Firma
Pendiente de verificación por un compañero real.	Pendiente de firma real.

Cierre seguro del puesto

- Copie la dirección de su repositorio o respalde su carpeta en su propio dispositivo.

Llenado de ejemplo: resultados, bitácoras y casillas simulados; confirmar con la ejecución real.

- Cierre su sesión de GitHub en el navegador.
- Cierre su sesión en Visual Studio Code, si inició alguna.
- Cierre la ventana del navegador, especialmente si era una ventana privada.
- No elimine credenciales guardadas de Windows por su cuenta: eso lo hace el docente si corresponde.
- Si aparece una cuenta ajena en el equipo, deténgase y avise.

Evidencia de la sesión

Evidencia	Listo
Repositorio local de práctica creado	☒
Archivo README.md con el contenido base	☒
Nombre y correo configurados dentro del repositorio	☒
Primer commit registrado	☒
Historial visible con git log --oneline	☒
Segundo commit registrado (deseable)	☒
Repositorio remoto privado publicado (deseable, solo con cuenta y conexión)	☐ N/A

Registro de dificultades

Etapa	Problema	¿Se resolvió?
5	Faltó identidad al hacer commit; configurar user.name y user.email.	Sí, en modelo
6	Visor del historial abierto; salir con q.	Sí, en modelo
7	Publicación opcional omitida; se conserva la evidencia local.	No aplica

Qué se versiona y qué no

Sirve para versionar	No suba a un repositorio
Código, HTML, CSS y JavaScript	Archivos grandes de audio y video
Documentación, guiones y textos	Contraseñas, claves y credenciales
Diagramas, prototipos y bocetos	Datos reales de una organización
Recursos gráficos pequeños	Carpetas de dependencias descargadas

Para archivos multimedia grandes existen extensiones específicas de Git. Se revisarán más adelante solo si su proyecto lo necesita.

Para el miércoles 12 de agosto

- Traiga identificados dos posibles casos organizacionales para el proyecto integrador: una necesidad concreta, con usuarios identificables.
- Ese día se conforman los equipos definitivos y cada equipo creará el repositorio del proyecto, con el mismo flujo que practicó hoy.
- El repositorio de hoy es individual y de práctica: no es todavía el del proyecto.

Llenado de ejemplo: resultados, bitácoras y casillas simulados; confirmar con la ejecución real.